Université Saint-Joseph de Beyrouth
Faculté des sciences

Relational Database **Project**

REPORT

Yara AL GHADDAF

Table of **contents**

01 Introduction

02 Database Implementation

03 SQL Queries

04 Output Analysis

05 Future Enhancements

06 Conclusion

01

Introduction

01

Presentation

Introduction

This project involves creating a **database** to manage the operations of a **learning center**. Databases help store, organize, and manage data efficiently, and a **Database Management System (DBMS)** makes it easy to retrieve, update, and secure this data.

Introduction

The project focuses on creating a database to manage the core functions of a learning center (students, courses, enrollments...)

It showcases how SQL can be used to store, manage, and retrieve data effectively.

EDRAWMAX
ER DIAGRAM

Wondershare
EdrawMax

ER DIAGRAM LINK
https://www.edrawmax.com/online/share.html?code=214d5a30b70411ef8eb50a54be41f961

01

Explanation of Entities and Relationships

ER DIAGRAM

TABLES:

-Students studentId PK
-Courses courseId PK
-Enrollments enrollmentId PK , studentId and courseId FK
-Instructors instructorId PK
-Courses_Instructors course_instructorId PK , instructorId and courseId FK
-Schedules scheduleId PK , classroomId,instructorId and courseId FK
-Attendances attendanceId PK , studentId and scheduleId FK
-Payments paymentId PK , studentId and coursed FK
-Classrooms classroomId PK
-Assignments assignmentId PK , courseId FK
-Submissions submissionId PK , assignmentId and studentId FK
-Materials materialId PK , courseId FK
-Feedbacks feedbackId PK , studentId and courseId FK

ER DIAGRAM

RELATIONSHIPS:

1)Students-Courses
Many To Many relationship linked with the table –Enrollments
A student can be enrolled in many courses and a course can have many students enrolled to.

2)Courses-Instructors
Many To Many relationship linked with the table –Courses_Instructors
A course can be taught by many instructors and an instructor can teach many courses.

3)Students_Schedules
Many To Many relationship linked with the table –Attendances
A student can have many schedule and a schedule can belong to many students.

ER DIAGRAM

RELATIONSHIPS:

4)Feedbacks-Courses
Many To One relationship
A feedback can be given by one course through one student, but a course has many feedbacks.

5)Courses-Schedules
One To Many relationship
A course have many schedules, but a schedule belongs to one course.

6)Courses-Materials
One To Many relationship
A course has many materials, but a material belongs to one course.

ER DIAGRAM

RELATIONSHIPS:

10)Students-Assignments
Many To Many relationship linked with the table –Submissions
A student can do many assignments and an assignment can be given to many students.

11)Instructors-Schedules
One To Many
An instructor can have many schedules, but a schedule belongs to one instructor.

12)Schedules-Classrooms
Many To One relationship
A schedule belongs to one classroom, but a classroom can have many schedules.

ER DIAGRAM

RELATIONSHIPS:

13)Courses-Assignments
One To Many relationship
A course can have many assignments, but an assignment is related to a course.

02

Database Implementation

SQL SERVER MANAGEMENT SYSTEM DATABASE

02

Table Creation (CREATE Statements)

LEGEND

TABLE NAME
COLUMNS
TYPE
AUTO_INCREMENT
STARTING FROM ONE

TABLE CREATION

SAMPLE 1

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY IDENTITY(1,1),
    FirstName NVARCHAR(50),
    LastName NVARCHAR(50),
    Email VARCHAR(100),
    PhoneNumber VARCHAR(20),
    DateOfBirth DATE
);
```

TABLE CREATION

SAMPLE 2

```
CREATE TABLE Schedules (
    ScheduleID INT PRIMARY KEY IDENTITY(1,1),
    CourseID INT,
    InstructorID INT,
    ClassroomID INT,
    StartTime DATETIME,
    EndTime TIME,
    Days VARCHAR(50),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID),
    FOREIGN KEY (InstructorID) REFERENCES Instructors(InstructorID),
    FOREIGN KEY (ClassroomID) REFERENCES Classrooms(ClassroomID)
);
```

TABLE CREATION

SAMPLE 3

```
CREATE TABLE Submissions (
    SubmissionID INT PRIMARY KEY IDENTITY(1,1),
    AssignmentID INT,
    StudentID INT,
    Grade INT,
    FOREIGN KEY (AssignmentID) REFERENCES
Assignments(AssignmentID),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
```

Key Constraints Explanation

01

PK

A unique identifier for each record in a table. It ensures no duplicate values and cannot be NULL.

02

FK

A column that links two tables, referencing the Primary Key of another table to establish a relationship.

DATA TYPES EXPLANATION

01

DATETIME

Used to store large blocks of text, such as descriptions or notes

02

TEXT

Used to store large blocks of text, such as descriptions or notes.

03

DATE

Stores only the date in YYYY-MM-DD format.

DATA TYPES EXPLANATION

04

VARCHAR

Stores text of variable length with a limit on the number of character

05

NVARCHAR

Stores text of variable length with a limit and can store a wider range of characters, including non-English characters and symbols.

06

INT

Used to store whole numbers (integers) without decimal points, such as 1, 100, or -50. It is commonly used for IDs and counters.

DATA TYPES EXPLANATION

07

DECIMAL

Stores numbers with up to P digits, N of which are after the decimal

02

Data Insertion (INSERT Statements)

DATA INSERTION

SAMPLE 1

```
INSERT INTO Students (FirstName, LastName, Email, PhoneNumber, DateOfBirth)
VALUES
('Alice', 'Smith', 'alice.smith@gmail.com', '71501442', '2005-05-14'),
('Bob', 'Johnson', 'bob.johnson@gmail.com', '03815618', '2004-07-23'),
('Charlie', 'Brown', 'charlie.brown@gmail.com', '71289988', '2005-09-12'),
('David', 'Lee', 'david.lee@gmail.com', '81619153', '2004-01-15'),
('Emma', 'Jones', 'emma.jones@gmail.com', '78892328', '2003-11-03');
```

This line, inserts Alice's data into the Students table, including her first name, last name, email, phone number, and date of birth.

DATA INSERTION

SAMPLE 2

```
INSERT INTO Schedules (CourseID, InstructorID, ClassroomID, StartTime, EndTime, Days)
VALUES
(1, 1, 1, '09:00:00', '10:30:00', Monday, Wednesday'),
(2, 2, 2, '09:00:00', '10:30:00', 'Monday, Tuesday'),
(2, 1, 1, '14:00:00', '15:30:00', 'Monday, Tuesday'),
(3, 3, 3, '11:00:00', '12:30:00', 'Thursday'),
(4, 4, 4, '13:00:00', '14:30:00', 'Thursday'),
(5, 5, 5, '10:00:00', '11:30:00', 'Friday');
```

This line schedules the "Introduction to Data Science" course with instructor "John Doe" in classroom "A101" on Monday and Wednesday from 9:00 AM to 10:30 AM.

DATA INSERTION

SAMPLE 3

```
INSERT INTO Attendances (StudentID, ScheduleID, Status)
VALUES
(1, 1, 'Present'),
(2, 2, 'Present'),
(1, 3, 'Present'),
(3, 4, 'Absent'),
(4, 5, 'Present'),
(5, 6, 'Present');
```

This line records Alice's attendance as "Present" for the "Introduction to Data Science" class, scheduled in the first schedule entry (ScheduleID 1).

02

Data Retrieval (SELECT Statements)

DATA RETRIEVAL

SAMPLE 1

('Alice', 'Smith', 'alice.smith@gmail.com', '71501442', '2005-05-14'),

SELECT* FROM STUDENTS;

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	alice.smith@gmail.com	71501442	2005-05-14
2	2	Bob	Johnson	bob.johnson@gmail.com	03815618	2004-07-23
3	3	Charlie	Brown	charlie.brown@gmail.com	71289988	2005-09-12
4	4	David	Lee	david.lee@gmail.com	81619153	2004-01-15
5	5	Emma	Jones	emma.jones@gmail.com	78892328	2003-11-03

For each INSERT INTO Students row, we capture the details of each student, including their first name, last name, email, phone number, and date of birth. This table helps store basic information about students attending the center.

DATA RETRIEVAL

SAMPLE 2

(1, 1, 1, '09:00:00', '10:30:00', Monday, Wednesday'),

SELECT* FROM SCHEDULES;

	ScheduleID	CourseID	InstructorID	ClassroomID	StartTime	EndTime	Days
1	1	1	1	1	1900-01-01 09:00:00.000	10:30:00.0000000	Monday, Wednesday
2	2	2	2	2	1900-01-01 09:00:00.000	10:30:00.0000000	Monday, Tuesday
3	3	2	1	1	1900-01-01 14:00:00.000	15:30:00.0000000	Monday, Tuesday
4	4	3	3	3	1900-01-01 11:00:00.000	12:30:00.0000000	Thursday
5	5	4	4	4	1900-01-01 13:00:00.000	14:30:00.0000000	Thursday
6	6	5	5	5	1900-01-01 10:00:00.000	11:30:00.0000000	Friday

For each INSERT INTO Schedules row, we capture which course is taught by which instructor in a specific classroom, along with the start and end times and the days of the week the class is held. This table helps organize and track the scheduling of classes.

DATA RETRIEVAL

SAMPLE 3

(1, 1, 'Present'),

SELECT * FROM Attendances;

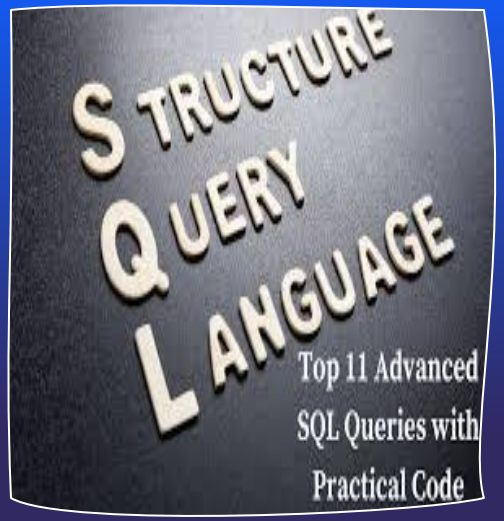
	AttendanceID	StudentID	ScheduleID	Status
1	1	1	1	Present
2	2	2	2	Present
3	3	1	3	Present
4	4	3	4	Absent
5	5	4	5	Present
6	6	5	6	Present

For each INSERT INTO Attendances row, we capture which student attended which class session and their attendance status. The table helps track student participation in scheduled classes.

03

SQL QUERIES

The queries in this project were designed to interact with a learning center database, covering core functionalities like retrieving, updating, and analyzing data.
They are categorized as basic, intermediate, and advanced queries based on their complexity and purpose.
These queries handle operations such as managing students, courses, enrollments, instructors, feedback, payments, attendance, and schedules.



QUERIES

1)Basic CRUD Operations such as inserting, updating, deleting, and selecting data from multiple tables.

2)Filtering and Sorting data with the WHERE, LIKE, BETWEEN, and ORDER BY clauses.

3)Aggregate Functions such as COUNT, AVG, SUM, and MAX to summarize and group data.

4)Joins like INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN to combine data from multiple tables.

5)Subqueries for advanced filtering and data retrieval.

6)Complex Queries that leverage conditional statements (CASE) and aggregation across multiple tables.

7)Triggers and Constraints to maintain data integrity and track changes in the database (e.g., logging payment and enrollment changes).

04

OUTPUT Analysis

Basic Queries OUTPUT and Explanation

SAMPLE 1: SIMPLE SELECTION

SELECT FirstName, LastName FROM Students;

This query retrieves the FirstName and LastName columns from the Students table. It does not apply any filters or conditions, meaning it will return these two columns for all rows in the Students table.

	FirstName	LastName
1	Alice	Smith
2	Bob	Johnson
3	Charlie	Brown
4	David	Lee
5	Emma	Jones

NB: *=ALL

Basic Queries OUTPUT and Explanation

SAMPLE 2: FILTERING

SELECT * FROM Students WHERE DateOfBirth > '2004-01-01';

This query selects all columns from the Students table but filters the rows based on the condition DateOfBirth > '2004-01-01'. Only students born after January 1, 2004, will appear in the result.

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	alice.smith@gmail.com	71501442	2005-05-14
2	2	Bob	Johnson	bob.johnson@gmail.com	03815618	2004-07-23
3	3	Charlie	Brown	charlie.brown@gmail.com	71289988	2005-09-12
4	4	David	Lee	david.lee@gmail.com	81619153	2004-01-15

NB: *=ALL

Basic Queries OUTPUT and Explanation

SAMPLE 3: SORTING

SELECT * FROM Students ORDER BY LastName DESC;

This query selects all columns from the Students table and sorts the results by the LastName column in descending order (Z to A).

NB: *=ALL

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	alice.smith@gmail.com	71501442	2005-05-14
2	4	David	Lee	david.lee@gmail.com	81619153	2004-01-15
3	5	Emma	Jones	emma.jones@gmail....	78892328	2003-11-03
4	2	Bob	Johnson	bob.johnson@gmail....	03815618	2004-07-23
5	3	Charlie	Brown	charlie.brown@gmail....	71289988	2005-09-12

Basic Queries OUTPUT and Explanation

SAMPLE 4: AGGREGATE FUNCTION

SELECT COUNT(*) AS TotalStudents FROM Students;

This query uses the COUNT() aggregate function to calculate the total number of rows (students) in the Students table. The result is returned as a single-row output with a column labeled TotalStudents.

	TotalStudents
1	5

NB: *=ALL

Intermediate Queries OUTPUT and Explanation

SAMPLE 1: UPDATING DATA

```
UPDATE Students
SET Email = 'new.email@example.com'
WHERE StudentID = 1;
```

This query updates the Email column of the student table with StudentID = 1 to 'new.email@example.com'.

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	new.email@example.com	71501442	2005-05-14
2	2	Bob	Johnson	bob.johnson@gmail.com	03815618	2004-07-23
3	3	Charlie	Brown	charlie.brown@gmail.com	71289988	2005-09-12
4	4	David	Lee	david.lee@gmail.com	81619153	2004-01-15

Intermediate Queries OUTPUT and Explanation

SAMPLE 2: JOIN OPERATION

```
SELECT Students.FirstName, Students.LastName, Courses.CourseName
FROM Students
JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
JOIN Courses ON Enrollments.CourseID = Courses.CourseID;
```

This query retrieves the first name and last name of students along with the names of the courses they are enrolled in. It performs two inner joins

	FirstName	LastName	CourseName
1	Alice	Smith	Introduction to Data Science
2	Alice	Smith	Advanced Mathematics
3	Bob	Johnson	Advanced Mathematics
4	Charlie	Brown	Organic Chemistry
5	David	Lee	English001
6	Emma	Jones	Economics for Beginners

Intermediate Queries OUTPUT and Explanation

SAMPLE 3: GROUPING DATA

```
SELECT Specialization, AVG(Salary) AS AvgSalary
FROM Instructors
GROUP BY Specialization;
```

This query calculates the average salary (AVG(Salary)) for instructors grouped by their Specialization.

	Specialization	AvgSalary
1	Biochemistry	5700.000000
2	Data Science	7000.000000
3	Economics	2900.000000
4	Language	4500.000000
5	Mathematics	4800.000000

Intermediate Queries OUTPUT and Explanation

SAMPLE 4:CONDITIONAL LOGIC

```
SELECT CASE
        WHEN Capacity < 20 THEN 'Small'
        WHEN Capacity BETWEEN 20 AND 30 THEN 'Medium'
        ELSE 'Large'
      END AS CapacityCategory,
      COUNT(*) AS RoomCount
FROM Classrooms
GROUP BY CASE
        WHEN Capacity < 20 THEN 'Small'
        WHEN Capacity BETWEEN 20 AND 30 THEN 'Medium'
        ELSE 'Large'
      END;
```

	CapacityCategory	RoomCount
1	Large	3
2	Medium	2

This query categorizes classrooms into three capacity categories (Small, Medium, Large) based on their Capacity. It counts the number of classrooms in each category.

Intermediate Queries OUTPUT and Explanation

SAMPLE 5: SUBQUERIES

```
SELECT Students.FirstName, Students.LastName
FROM Students
WHERE NOT EXISTS (
    SELECT 1
    FROM Enrollments
    WHERE Enrollments.StudentID = Students.StudentID
);
```

FirstName	LastName

This query retrieves the names of students who are not enrolled in any courses. It uses a subquery in the NOT EXISTS clause to check for students whose StudentID does not appear in the Enrollments table.

Intermediate Queries OUTPUT and Explanation

SAMPLE 6: UNION OPERATION

```
SELECT StudentID
FROM Enrollments
WHERE CourseID = 1
UNION
SELECT StudentID
FROM Enrollments
WHERE CourseID = 2;
```

	StudentID
1	1
2	2

This query combines the StudentIDs of students enrolled in either Course 1 or Course 2 without including duplicates. The UNION operator ensures the results are distinct

Advanced Queries OUTPUT and Explanation

SAMPLE 1: NESTED SUBQUERIES

```
SELECT CourseName
FROM Courses
WHERE Price < ALL (
    SELECT Salary
    FROM Instructors
);
```

	CourseName
1	Introduction to Data Science
2	Advanced Mathematics
3	Organic Chemistry
4	English001
5	Economics for Beginners

This query retrieves the names of courses whose price is lower than the salary of all instructors. It uses a nested subquery

Advanced Queries OUTPUT and Explanation

SAMPLE 2: WINDOW FUNCTIONS

```
SELECT s.FirstName, s.LastName, SUM(p.Amount) AS TotalPaid,
       RANK() OVER (ORDER BY SUM(p.Amount) DESC) AS Rank
FROM Payments p
JOIN Students s ON p.StudentID = s.StudentID
GROUP BY s.StudentID, s.FirstName, s.LastName;
```

This query calculates the total payments made by each student and ranks them based on the amount paid, using the RANK() window function.

	FirstName	LastName	TotalPaid	Rank
1	Alice	Smith	3100.00	1
2	Charlie	Brown	2700.00	2
3	Bob	Johnson	1600.00	3
4	Emma	Jones	1550.00	4
5	David	Lee	950.00	5

Advanced Queries OUTPUT and Explanation

SAMPLE 3: COMMON TABLE EXPRESSIONS (CTEs)

```
WITH CoursePopularity AS (
    SELECT CourseID, COUNT(StudentID) AS EnrolledStudents
    FROM Enrollments
    GROUP BY CourseID
)
SELECT c.CourseName, cp.EnrolledStudents
FROM Courses c
JOIN CoursePopularity cp ON c.CourseID = cp.CourseID
WHERE cp.EnrolledStudents > 2;
```

CourseName	EnrolledStudents

This query uses a Common Table Expression (CTE) to calculate the number of students enrolled in each course and then retrieves course names where more than two students are enrolled.

Advanced Queries OUTPUT and Explanation

SAMPLE 4: HAVING CLAUSE

```
SELECT CourseID, COUNT(StudentID) AS EnrolledStudents
FROM Enrollments
GROUP BY CourseID
HAVING COUNT(StudentID) > 1;
```

This query retrieves courses that have more than one student enrolled, using the HAVING clause.

	CourseID	EnrolledStudents
1	2	2

Advanced Queries OUTPUT and Explanation

SAMPLE 5: FEEDBACK ANALYSIS

```
SELECT c.CourseName,
       AVG(CAST(f.Rating AS FLOAT)) AS AvgRating,
       COUNT(f.FeedbackID) AS TotalFeedbacks
FROM Feedbacks f
INNER JOIN Courses c ON f.CourseID = c.CourseID
GROUP BY c.CourseName;
```

This query calculates the average rating and the total number of feedback entries for each course.

	CourseName	AvgRating	TotalFeedbacks
1	Advanced Mathematics	4	2
2	Economics for Beginners	4	1
3	English001	1	1
4	Introduction to Data Sc...	5	1
5	Organic Chemistry	3	1

Complex Queries OUTPUT and Explanation

SAMPLE 1: ADVANCED JOINS (Multiple Joins with Self Join)

```
SELECT S1.FirstName AS Student1, S1.LastName AS LastName1,
       S2.FirstName AS Student2, S2.LastName AS LastName2,
       Courses.CourseName
FROM Students S1
JOIN Enrollments E1 ON S1.StudentID = E1.StudentID
JOIN Enrollments E2 ON E1.CourseID = E2.CourseID
JOIN Students S2 ON E2.StudentID = S2.StudentID
JOIN Courses ON E1.CourseID = Courses.CourseID
WHERE S1.StudentID < S2.StudentID;
```

	Student1	LastName1	Student2	LastName2	CourseName
1	Alice	Smith	Bob	Johnson	Advanced Mathematics

This query retrieves pairs of students enrolled in the same course. It uses multiple joins, including a self-join on the Students table to pair students together.

Complex Queries OUTPUT and Explanation

SAMPLE 2: TRIGGERS

```
CREATE TRIGGER LogPaymentStatusChange
ON Payments
AFTER UPDATE
AS
BEGIN
    INSERT INTO PaymentStatusLog (PaymentID, OldStatus, NewStatus)
    SELECT d.PaymentID, d.Status AS OldStatus, i.Status AS NewStatus
    FROM DELETED d
    JOIN INSERTED i ON d.PaymentID = i.PaymentID;
END;
```

This trigger logs changes to payment statuses whenever the Payments table is updated. It uses the INSERTED and DELETED tables to track changes.

05

Future Enhacements

05

General Idea

The enhancements aim to improve the learning center system by adding features that enhance accessibility, personalization, and efficiency. These upgrades are designed to create a more effective and engaging learning experience for students and instructors alike.

05

Improvements

Future

Access to External Learning Resources

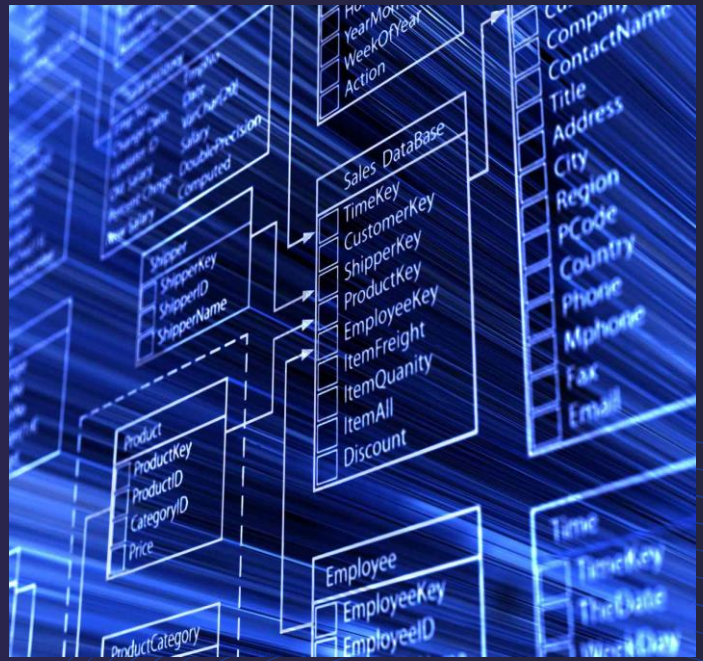
Link the platform to online courses and educational videos, giving students access to more learning materials within the learning center.

Automated Progress Tracking and Feedback

Automatically track student progress and provide personalized feedback to highlight strengths and improvement areas, helping students focus on their learning needs.

06

Conclusion

In conclusion, this learning center system successfully manages various aspects of the educational environment, from course enrollment and scheduling to attendance tracking and communication between students and instructors.

The database for the Learning Center project is designed to be efficient, secure, and scalable. It ensures data accuracy through primary and foreign keys, linking all tables such as students, courses, and schedules correctly..